

Fase 14 — Fragmentación Vertical

Materia

BDD

Continuación directa de la Fase 13. Los cinco nodos del laboratorio tienen sus snapshots `fase13-completa` tomados: `bdd-nodo01` es el maestro de replicación física, `bdd-nodo02` su esclavo con `read_only = ON`, `bdd-nodo03` el nodo multi-maestro, `bdd-nodo04` almacena el fragmento horizontal A (regiones norte y este de `clientes`, `pedidos` y `detalle_pedidos`) y `bdd-nodo05` el fragmento horizontal B (regiones sur y oeste). Ambos shards tienen además el catálogo completo de `productos` como tabla temporal íntegra.

En esta fase se realizan dos tareas complementarias: los diez productos se distribuyen **verticalmente** entre `nodo04` (`v_productos_basico`, columnas operacionales) y `nodo05` (`v_productos_detalle`, columnas de detalle de gran volumen), y se crea `bdd-nodo06` como **coordinador de consultas distribuidas** usando el motor **Spider** de MariaDB.

Al finalizar, el cliente puede conectarse a `nodo06` y ejecutar consultas globales —accediendo a datos físicamente distribuidos en `nodo04` y `nodo05`— sin conocer la distribución subyacente: primera demostración completa de **transparencia de fragmentación** y **transparencia de ubicación** en el laboratorio.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Crear y poblar los dos **fragmentos verticales** de `productos` en los nodos de sharding, aplicando el plan definido en el Documento de Diseño Distribuido (DDD) de la Fase 9 y verificando las tres condiciones de correctitud: completitud, disjunción de columnas no clave y reconstrucción por `JOIN`.
2. Explicar qué es el motor **Spider** de MariaDB, cómo actúa como proxy transparente hacia tablas remotas y en qué se diferencia del motor `FEDERATED` (descontinuado).
3. Provisionar `bdd-nodo06` como clon enlazado del snapshot `fase07-completa` de `nodo01`, configurar su hostname, IP estática `192.168.56.106` y `server_id = 6`.

4. Instalar y habilitar el plugin Spider con `INSTALL SONAME 'ha_spider'`, verificando que el motor queda `ACTIVE` en `information_schema.PLUGINS` y que sus tablas de sistema `mysql.spider_*` existen correctamente.
5. Registrar los nodos remotos con `CREATE SERVER`, explicando las ventajas de este enfoque frente a incrustar credenciales directamente en el `COMMENT` de cada tabla Spider.
6. Crear **tablas Spider particionadas** para `clientes` y `pedidos` usando `PARTITION BY LIST COLUMNS (region)`, conectando el concepto de poda de particiones aprendido en la Fase 12 con el enrutamiento automático hacia el shard correcto.
7. Diseñar y justificar la solución para `detalle_pedidos` (sin columna `region`) usando dos tablas Spider simples más una `VIEW UNION ALL`, explicando por qué no es posible usar `PARTITION BY LIST COLUMNS` para esta tabla.
8. Crear las tablas Spider para los fragmentos verticales de `productos` y la `VIEW productos` que los reconstruye por `JOIN`, logrando transparencia de fragmentación vertical.
9. Ejecutar y analizar consultas distribuidas desde nodo06 que demuestren poda de particiones (acceso a un solo shard), `UNION ALL` implícito (ambos shards) y `JOIN` distribuido entre fragmentos verticales.
10. Tomar el snapshot `fase14-completa` en los seis nodos del laboratorio.

B. Conceptos teóricos necesarios

1. Revisión: fragmentación vertical y condiciones de correctitud.

La fragmentación vertical divide las **columnas** de una relación `R` en proyecciones disjuntas, cada una acompañada de la clave primaria:

Text

$$V_i = \pi(C_i \cup \{PK\}) (R)$$

Las tres condiciones de correctitud verificadas analíticamente en la Fase 9 se trasladan ahora a nodos físicos separados:

- **Completitud:** todos los productos deben existir en `V_basico` (nodo04) y en `V_detalle` (nodo05); ninguna fila puede perderse en ninguno de los dos fragmentos.
- **Disjunción de columnas no clave:** `{nombre, categoria, precio, stock, fecha_creacion}`
`n {descripcion, ficha_tecnica, imagen_url, peso_kg} = ∅`. La PK (`id`) y la clave alternativa (`sku`) se duplican intencionalmente en ambos fragmentos para permitir la reconstrucción mediante `JOIN`.
- **Reconstrucción:** `JOIN` por `id` entre los dos fragmentos reproduce la relación `productos` completa sin pérdida de atributos.

2. El motor Spider de MariaDB.

Spider es un motor de almacenamiento nativo de MariaDB —integrado desde la versión 10.0, estable desde 10.3— que permite crear tablas cuyos datos residen en uno o varios servidores MariaDB o MySQL remotos. Para el cliente, las tablas Spider son indistinguibles de tablas InnoDB locales: mismo SQL estándar, mismos tipos de datos, mismas operaciones DML.

Característica	FEDERATED (descontinuado)	Spider
Particionamiento entre nodos	No	Sí (RANGE, LIST, HASH, KEY)
JOIN distribuido	No	Sí (coordinador recombina resultados)
Transacciones XA	No	Soporte parcial
Mantenimiento activo	Abandonado	Activo en MariaDB 10.x
Tablas de sistema propias	No	Sí (<code>mysql.spider_*</code>)

Internamente, Spider traduce el SQL del cliente en llamadas TCP al servidor remoto usando el protocolo binario de MariaDB. Para un `SELECT` con filtro sobre la columna de particionamiento, Spider envía la subconsulta únicamente al shard relevante (**poda remota**). Para un `SELECT` sin filtro, consulta todos los shards y combina los resultados en el coordinador mediante `UNION ALL`.

3. Modos de operación de Spider en este laboratorio.

Modo	Descripción	Aplicación en Fase 14
Simple	Una tabla Spider apunta a una sola tabla remota en un nodo	<code>v_productos_basico</code> , <code>v_productos_detalle</code> , <code>spider_detalle_nodo04</code> , <code>spider_detalle_nodo05</code>
Horizontal	<code>PARTITION BY</code> <code>LIST COLUMNS</code> enruta filas según el valor de la columna de partición hacia distintos shards	<code>clientes</code> , <code>pedidos</code> (particionadas por <code>region</code>)
Vertical	Dos tablas Spider simples más una <code>VIEW</code> <code>JOIN</code> reconstruyen columnas distribuidas entre dos nodos	<code>VIEW</code> <code>productos</code> = JOIN de V_basico y V_detalle

4. `CREATE SERVER` : definición de conexiones remotas reutilizables.

`CREATE SERVER` almacena un conjunto de parámetros de conexión bajo un nombre simbólico. Las tablas Spider referencian ese nombre en el `COMMENT` en lugar de repetir IP, puerto, usuario y contraseña en cada definición:

SQL

```
CREATE SERVER srv_nodo04
  FOREIGN DATA WRAPPER mysql
  OPTIONS (
    HOST      '192.168.56.104',
    PORT      3306,
    DATABASE  'lab_bdd',
    USER      'spider_user',
    PASSWORD  'Spider_2025!'
  );
-- Uso posterior en tabla Spider:
CREATE TABLE t (...) ENGINE = SPIDER
  COMMENT = 'server "srv_nodo04", table "nombre_tabla_remota";
```

Los servidores se persisten en `mysql.servers` y se eliminan con `DROP SERVER nombre`.

5. Tablas Spider particionadas: conexión con la Fase 12.

Las tablas Spider `clientes` y `pedidos` del coordinador usan `PARTITION BY LIST COLUMNS (region)`. El motor aplica **poda de particiones remota**: ante `WHERE region = 'norte'`, Spider envía la subconsulta solo al shard que contiene esa región. Este es exactamente el mismo mecanismo de la Fase 12 extendido a nodos físicamente distintos.

La clave primaria de la tabla Spider particionada debe incluir la columna de particionamiento, requerimiento idéntico al del particionamiento nativo de la Fase 12:

SQL

```
PRIMARY KEY (id, region)  -- region obligatoria en PK para
Spider particionado
```

La tabla remota en nodo04 puede mantener `PRIMARY KEY (id)` sin cambios; esta discrepancia en la definición de PK es permitida por Spider y no genera conflictos en la práctica.

6. Por qué `detalle_pedidos` no puede usar `PARTITION BY LIST COLUMNS (region)`.

`detalle_pedidos` no tiene columna `region` . Spider necesita que la columna de particionamiento exista en la tabla para poder aplicar la poda. Sin `region` , no es posible usar el mismo esquema de particionamiento que `clientes` y `pedidos` . La solución adoptada es crear una tabla Spider simple por shard y unir las mediante una `VIEW UNION ALL` , que Spider resuelve consultando ambos nodos y combinando los resultados en el coordinador.

7. Transparencia de distribución lograda al finalizar la fase.

Tipo de transparencia	Mecanismo implementado	Primera fase de verificación
Fragmentación	El cliente consulta <code>clientes</code> , <code>productos</code> , etc. sin saber que los datos están divididos entre nodos	Fase 14
Ubicación	El cliente conecta siempre a <code>192.168.56.106</code> ; ignora la existencia de nodo04 y nodo05	Fase 14
Replicación	Maestro-esclavo transparente para el cliente de nodo01	Fase 10
Concurrencia	MVCC de InnoDB; pruebas con transacciones concurrentes	Fase 17

C. Prerrequisitos

Antes de iniciar esta fase, verificar que se cumplen **todos** los puntos siguientes:

Estado de las máquinas virtuales:

Nodo	Snapshot requerido	Estado funcional esperado
<code>bdd-nodo01</code>	<code>fase13-completa</code>	MAESTRO de replicación; <code>lab_bdd</code> con 20/10/20/36 filas
<code>bdd-nodo02</code>	<code>fase13-completa</code>	ESCLAVO; <code>read_only = ON</code>
<code>bdd-nodo03</code>	<code>fase13-completa</code>	MULTI-MAESTRO; réplica bidireccional con nodo01
<code>bdd-nodo04</code>	<code>fase13-completa</code>	SHARD-A: <code>clientes</code> (10 filas norte+este), <code>pedidos</code> (10), <code>detalle_pedidos</code> (~18), <code>productos</code> (10, completo); <code>server_id = 4</code> ; <code>bind-address = 0.0.0.0</code>
<code>bdd-nodo05</code>	<code>fase13-completa</code>	SHARD-B: <code>clientes</code> (10 filas sur+oeste), <code>pedidos</code> (10), <code>detalle_pedidos</code> (~18), <code>productos</code> (10, completo); <code>server_id = 5</code> ; <code>bind-address = 0.0.0.0</code>

bdd-nodo06	—	No existe aún; se crea en esta fase
------------	---	---

Conocimiento técnico requerido:

- Fragmentación vertical: columnas operacionales vs. columnas de detalle (`v_basico` / `v_detalle`) y sus predicados de correctitud, tal como se diseñaron en la Fase 9 (DDD, sección 4).
- Particionamiento nativo `LIST COLUMNS` de MariaDB (Fase 12): los mismos predicados y el mismo mecanismo de poda de particiones se usan ahora en tablas Spider del coordinador.
- Creación de clones enlazados de VirtualBox y reconfiguración de hostname e IP (Fase 13, pasos D.2–D.5).

Recursos de host necesarios:

- Al menos **1 GB de RAM adicional** libre para nodo06 (1 536 MB asignados).
- Al menos **15 GB de disco libre** para el disco delta del clon enlazado.
- La estructura `C:\LabBDD\` con sus subcarpetas (creada en la Fase 0).

Archivos de referencia:

- `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` — secciones 3 (fragmentación horizontal) y 4 (fragmentación vertical) son la especificación técnica que guía esta fase.

D. Procedimiento paso a paso

Arquitectura al inicio de la Fase 14

Text

```
bdd-nodo01 [MAESTRO      | 192.168.56.101] lab_bdd completo
(20/10/20/36 filas)
bdd-nodo02 [ESCLAVO      | 192.168.56.102] lab_bdd réplica, read_only = ON
bdd-nodo03 [MULTI-M.     | 192.168.56.103] lab_bdd réplica bidireccional
bdd-nodo04 [SHARD-A      | 192.168.56.104] clientes(10)
pedidos(10) detalle(~18)
                                productos(10, completo
- temporal)
bdd-nodo05 [SHARD-B      | 192.168.56.105] clientes(10)
pedidos(10) detalle(~18)
                                productos(10, completo
- temporal)
[bdd-nodo06 NO EXISTE AÚN]
```

Arquitectura al finalizar la Fase 14

Text

```
bdd-nodo04  [SHARD-A | 192.168.56.104]
            + v_productos_basico (id, sku, nombre, categoria, precio,
stock, fecha_creacion)
bdd-nodo05  [SHARD-B | 192.168.56.105]
            + v_productos_detalle (id, sku, descripcion, ficha_tecnica,
imagen_url, peso_kg)
bdd-nodo06  [COORDINADOR | 192.168.56.106]  server_id=6, log_bin=OFF,
Spider ACTIVE

            lab_bdd (solo metadatos Spider – sin filas de datos de negocio):
            └─ clientes                        [SPIDER PARTITION LIST region
→ nodo04/nodo05]
            └─ pedidos                        [SPIDER PARTITION LIST region
→ nodo04/nodo05]
            └─ spider_detalle_nodo04 [SPIDER simple
→ nodo04.detalle_pedidos]
            └─ spider_detalle_nodo05 [SPIDER simple
→ nodo05.detalle_pedidos]
            └─ v_productos_basico [SPIDER simple →
nodo04.v_productos_basico]
            └─ v_productos_detalle [SPIDER simple →
nodo05.v_productos_detalle]
            └─ VIEW detalle_pedidos [UNION ALL spider_detalle_nodo04
+ nodo05]
            └─ VIEW productos [JOIN v_productos_basico
⋈ v_productos_detalle]
```

Paso 1 — Iniciar nodo04 y nodo05; verificar el estado heredado de la Fase 13 (conteos de filas, `bind-address`, `server_id`).

Paso 2 — Crear el usuario `spider_user` en nodo04 y en nodo05, restringido a la IP futura de nodo06 (`192.168.56.106`).

Paso 3 — Crear la tabla `v_productos_basico` en nodo04 con las columnas operacionales y poblarla desde la tabla `productos` completa que ya existe en ese nodo.

Paso 4 — Crear la tabla `v_productos_detalle` en nodo05 con las columnas de detalle y poblarla desde la tabla `productos` completa en ese nodo.

Paso 5 — Verificar las tres condiciones de correctitud de la fragmentación vertical en nodo04 y nodo05 (completitud, disjunción de columnas, reconstrucción simulada).

Paso 6 — Apagar nodo04 y nodo05 antes de crear nodo06, para evitar conflicto de IP cuando el clon arranque temporalmente con la dirección `192.168.56.101`.

Paso 7 — Crear `bdd-nodo06` como clon enlazado de `bdd-nodo01` tomado desde el snapshot `fase07-completa` (MariaDB instalado, sin `lab_bdd`).

Paso 8 — Arrancar solo nodo06, configurar hostname (`bdd-nodo06`), IP estática (`192.168.56.106`) y `/etc/hosts` con las entradas de todos los nodos del laboratorio.

Paso 9 — Crear el archivo de configuración de MariaDB del coordinador:

```
server_id = 6 , log_bin = OFF , bind-address = 0.0.0.0 , skip_name_resolve = ON .
```

Desactivar cualquier archivo de configuración de replicación heredado del clon.

Paso 10 — Instalar el motor Spider en nodo06 con `INSTALL SONAME 'ha_spider'`. Verificar que el plugin queda `ACTIVE` y que las tablas de sistema `mysql.spider_*` fueron creadas.

Paso 11 — Arrancar nodo04 y nodo05 (ya en sus IPs `.104` y `.105`). Verificar conectividad TCP desde nodo06 hacia ambos shards en el puerto 3306.

Paso 12 — Registrar los servidores remotos en nodo06 con `CREATE SERVER srv_nodo04` y `CREATE SERVER srv_nodo05`.

Paso 13 — Crear la base de datos `lab_bdd` en nodo06 y las tablas Spider para los fragmentos horizontales: `clientes` y `pedidos` (particionadas por `region`), y los dos pares de tablas Spider para `detalle_pedidos` más su `VIEW UNION ALL`.

Paso 14 — Crear las tablas Spider `v_productos_basico` y `v_productos_detalle` en nodo06 y la `VIEW productos` que las reconstruye por `JOIN`.

Paso 15 — Ejecutar las consultas de verificación distribuida desde nodo06: acceso particionado a `clientes` (poda activa), ficha completa de producto (JOIN distribuido vertical) y consulta híbrida que combina fragmentación horizontal y vertical.

Paso 16 — Apagar todos los nodos y tomar el snapshot `fase14-completa` en los seis nodos del laboratorio.

E. Comandos completos

Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan en el prompt del motor MariaDB.

E.1 Iniciar nodo04 y nodo05; verificar el estado de la Fase 13 (host)

PowerShell

```
VBoxManage startvm "bdd-nodo04" --type headless
VBoxManage startvm "bdd-nodo05" --type headless
Start-Sleep -Seconds 35
```

Conectarse a ambos nodos:

PowerShell

```
# Terminal 1 — nodo04
ssh bddadmin@192.168.56.104
# Terminal 2 — nodo05
ssh bddadmin@192.168.56.105
```

Verificar el estado en **(VM — nodo04)**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Conteos heredados de la Fase 13
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM detalle_pedidos
UNION ALL
SELECT 'productos',        COUNT(*)          FROM productos;
-- Solo deben aparecer las regiones del fragmento A
SELECT DISTINCT region AS regiones_presentes FROM clientes ORDER BY region;
EOF
# Verificar parámetros clave de MariaDB
sudo mariadb -e "SHOW VARIABLES LIKE 'bind_address';"
sudo mariadb -e "SHOW VARIABLES LIKE 'server_id';"
```

Salida esperada en nodo04:

 Text

```
clientes          | 10
pedidos           | 10
detalle_pedidos   | ~18
productos         | 10
regiones_presentes: este, norte
bind_address:     0.0.0.0
server_id:        4
```

Repetir en nodo05 esperando `sur, oeste` y `server_id = 5`.

| Si `bind_address` devuelve `127.0.0.1` en algún shard, corregirlo antes de continuar:

 Bash

```
sudo sed -i 's/^bind-address.*/bind-address = 0.0.0.0/' \
/etc/mysql/mariadb.conf.d/50-server.cnf
sudo systemctl restart mariadb
sudo mariadb -e "SHOW VARIABLES LIKE 'bind_address';"
```

E.2 Crear el usuario `spider_user` en nodo04 y nodo05 (VM — nodo04 y nodo05)

Ejecutar en **nodo04**:

 Bash

```
sudo mariadb << 'EOF'
-- -----
-- Usuario de acceso remoto para Spider.
-- Host: IP exacta de bdd-nodo06 (principio de mínimo privilegio).
-- Privilegios: SELECT + escritura sobre lab_bdd.
-- -----
CREATE USER IF NOT EXISTS 'spider_user'@'192.168.56.106'
  IDENTIFIED BY 'Spider_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON lab_bdd.*
  TO 'spider_user'@'192.168.56.106';
FLUSH PRIVILEGES;
-- Verificar
SELECT User, Host FROM mysql.user WHERE User = 'spider_user';
SHOW GRANTS FOR 'spider_user'@'192.168.56.106';
EOF
```

Ejecutar el **mismo bloque** en **nodo05** sin modificaciones.

E.3 Crear `v_productos_basico` en nodo04 (VM — nodo04)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- FRAGMENTO VERTICAL A – Columnas operacionales (alta frecuencia)
-- Nodo destino: bdd-nodo04 (192.168.56.104)
-- Plan DDD Fase 9: id*, sku, nombre, categoria,
--                  precio, stock, fecha_creacion
-- Uso esperado: listados, búsquedas, cálculo de precios
-- =====
CREATE TABLE IF NOT EXISTS v_productos_basico (
    id            INT            NOT NULL AUTO_INCREMENT,
    sku           VARCHAR(50)    NOT NULL,
    nombre        VARCHAR(200)   NOT NULL,
    categoria     VARCHAR(100),
    precio        DECIMAL(10,2)  NOT NULL,
    stock         INT            DEFAULT 0,
    fecha_creacion DATETIME      DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_sku (sku)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4
  COLLATE = utf8mb4_unicode_ci
  COMMENT = 'Fragmento vertical basico de productos – Fase 14';
-- Poblar desde la tabla productos completa existente en este nodo
-- (cargada íntegramente como catálogo temporal en la Fase 13)
INSERT INTO v_productos_basico
    (id, sku, nombre, categoria, precio, stock, fecha_creacion)
SELECT id, sku, nombre, categoria, precio, stock, fecha_creacion
FROM productos;
-- Verificar la carga
SELECT 'Filas en v_productos_basico' AS metrica, COUNT(*) AS valor
FROM v_productos_basico;
-- Vista previa
SELECT id, sku, nombre, categoria, precio, stock
FROM v_productos_basico
ORDER BY id;
EOF
```


E.4 Crear `v_productos_detalle` en nodo05 (VM — nodo05)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- FRAGMENTO VERTICAL B - Columnas de detalle (baja frecuencia)
-- Nodo destino: bdd-nodo05 (192.168.56.105)
-- Plan DDD Fase 9: id*, sku, descripcion,
--                   ficha_tecnica, imagen_url, peso_kg
-- Uso esperado: pantalla de ficha completa de producto
-- =====
CREATE TABLE IF NOT EXISTS v_productos_detalle (
    id            INT            NOT NULL,
    sku           VARCHAR(50)    NOT NULL,
    descripcion   TEXT,
    ficha_tecnica TEXT,
    imagen_url    VARCHAR(500),
    peso_kg       DECIMAL(8,3),
    PRIMARY KEY (id),
    KEY idx_sku (sku)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4
  COLLATE = utf8mb4_unicode_ci
  COMMENT = 'Fragmento vertical detalle de productos - Fase 14';
-- Poblar desde la tabla productos completa en este nodo
INSERT INTO v_productos_detalle
    (id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg)
SELECT id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
FROM productos;
-- Verificar la carga
SELECT 'Filas en v_productos_detalle' AS metrica, COUNT(*) AS valor
FROM v_productos_detalle;
-- Vista previa con truncado de columnas TEXT
SELECT id, sku,
    LEFT(descripcion, 50) AS descripcion_preview,
    LEFT(ficha_tecnica, 50) AS ficha_preview,
    imagen_url,
    peso_kg
FROM v_productos_detalle
ORDER BY id;
EOF
```

E.5 Verificar correctitud de la fragmentación vertical (VM — nodo04 y nodo05)

Ejecutar en **nodo04**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- CONDICIÓN 1: COMPLETITUD
-- Todos los productos originales deben estar en v_productos_basico.
-- =====
SELECT 'Compleitud v_productos_basico'      AS condicion,
      (SELECT COUNT(*) FROM productos)      AS total_original,
      COUNT(*)                             AS filas_en_fragmento,
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA'
      END                                   AS resultado
FROM v_productos_basico;

-- =====
-- CONDICIÓN 2: DISJUNCIÓN DE COLUMNAS NO CLAVE
-- Las columnas TEXT de detalle NO deben existir en V_basico.
-- =====
SELECT 'Disjunción columnas V_productos_basico' AS condicion,
      CASE WHEN COUNT(*) = 0
            THEN 'OK — sin columnas de detalle'
            ELSE CONCAT('FALLA — columnas encontradas: ',
                        GROUP_CONCAT(COLUMN_NAME))
      END AS resultado
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'lab_bdd'
      AND TABLE_NAME = 'v_productos_basico'
      AND COLUMN_NAME IN ('descripcion', 'ficha_tecnica', 'imagen_url', 'peso_kg');
EOF
```

Ejecutar en **nodo05**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- COMPLETITUD de V_detalle
SELECT 'Compleitud V_productos_detalle' AS condicion,
      (SELECT COUNT(*) FROM productos) AS total_original,
      COUNT(*) AS filas_en_fragmento,
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA'
      END AS resultado
FROM v_productos_detalle;
-- DISYUNCIÓN: columnas operacionales NO deben estar en V_detalle
SELECT 'Disjunción columnas V_productos_detalle' AS condicion,
      CASE WHEN COUNT(*) = 0
            THEN 'OK - sin columnas operacionales'
            ELSE CONCAT('FALLA - columnas encontradas: ',
                       GROUP_CONCAT(COLUMN_NAME))
      END AS resultado
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'lab_bdd'
      AND TABLE_NAME = 'v_productos_detalle'
      AND COLUMN_NAME IN
('nombre', 'categoria', 'precio', 'stock', 'fecha_creacion');
-- RECONSTRUCCIÓN SIMULADA (local en nodo05):
-- JOIN entre v_productos_detalle y la tabla productos completa.
-- Verifica que todos los IDs tienen contraparte.
SELECT 'Reconstrucción (IDs coincidentes)' AS condicion,
      (SELECT COUNT(*) FROM productos) AS total_original,
      COUNT(*) AS filas_reconstruidas,
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA'
      END AS resultado
FROM v_productos_detalle d
JOIN productos p ON p.id = d.id;
EOF
```

Todos los resultados deben mostrar **OK** antes de continuar.

E.6 Apagar nodo04 y nodo05 (host)

El clon de nodo06 arrancará temporalmente con la IP `192.168.56.101` heredada del snapshot de nodo01. Para evitar conflictos ARP, todos los nodos deben estar apagados:

 Bash

```
# En la sesión SSH de nodo04
sudo poweroff
# En la sesión SSH de nodo05
sudo poweroff
```

Confirmar desde el host:

 PowerShell

```
VBoxManage list runningvms
# La salida debe estar vacía
```

E.7 Crear `bdd-nodo06` como clon enlazado (host)

 PowerShell

```
# Se clona desde fase07-completa de nodo01:
# MariaDB 10.11 instalado + utf8mb4 activo, SIN lab_bdd.
# Esta es la base más limpia para un coordinador que no almacena
# datos de negocio propios, solo metadatos Spider.
VBoxManage clonevm "bdd-nodo01" `
    --snapshot      "fase07-completa" `
    --options        linked `
    --name           "bdd-nodo06" `
    --basefolder     "C:\LabBDD\VMs" `
    --register
VBoxManage modifyvm "bdd-nodo06" `
    --description    "COORDINADOR Spider. Sin datos de negocio. IP 192.168.56.106.
server_id=6. Fase 14."
# Confirmar registro
VBoxManage showvminfo "bdd-nodo06" | findstr /I "Name State Memory"
```

Arrancar **solo nodo06** (los demás están apagados — sin conflicto de IP):

PowerShell

```
VBoxManage startvm "bdd-nodo06" --type headless
Start-Sleep -Seconds 35
# El clon tiene temporalmente la IP de nodo01 (192.168.56.101)
ssh bddadmin@192.168.56.101
```

E.8 Configurar hostname, IP y `/etc/hosts` en nodo06 (VM — nodo06)

Bash

```
# — 1. Cambiar el hostname —————
sudo hostnamectl set-hostname bdd-nodo06
sudo sed -i 's/bdd-nodo01/bdd-nodo06/g' /etc/hosts
hostname
# Esperado: bdd-nodo06
```

Bash

```
# — 2. Cambiar la IP de .101 a .106 —————
NETPLAN_FILE=$(ls /etc/netplan/*.yaml | head -1)
echo "Archivo Netplan detectado: $NETPLAN_FILE"
# Verificar contenido antes de editar
cat "$NETPLAN_FILE"
# Sustituir la dirección IP
sudo sed -i 's/192\.\168\.\56\.\101/192.168.56.106/g' "$NETPLAN_FILE"
# Confirmar el cambio
grep "192.168" "$NETPLAN_FILE"
# Aplicar — la sesión SSH se interrumpe en este punto
sudo netplan apply
```

La sesión SSH se pierde al cambiar la IP. Reconectar desde el host:

PowerShell

```
Start-Sleep -Seconds 8
ssh bddadmin@192.168.56.106
```

 Bash

```
# — 3. Verificar la nueva IP —————  
ip addr show | grep "192.168.56"  
# Esperado: inet 192.168.56.106/24  
ping -c 2 192.168.56.1    # host Windows – debe responder
```

 Bash

```
# — 4. Regenerar claves SSH (el clon heredó las de nodo01) —  
sudo rm -f /etc/ssh/ssh_host_*  
sudo ssh-keygen -A  
sudo systemctl restart ssh  
echo "Nuevas claves SSH generadas:"  
ls /etc/ssh/ssh_host_*.pub
```

 Bash

```
# — 5. Registrar todos los nodos en /etc/hosts —————  
sudo tee -a /etc/hosts > /dev/null << 'EOF'  
# Laboratorio BDD – nodos del cluster  
192.168.56.101 bdd-nodo01  
192.168.56.102 bdd-nodo02  
192.168.56.103 bdd-nodo03  
192.168.56.104 bdd-nodo04  
192.168.56.105 bdd-nodo05  
192.168.56.106 bdd-nodo06  
EOF  
cat /etc/hosts
```

E.9 Configurar MariaDB en nodo06 (VM — nodo06)

 Bash

```
# Desactivar archivos de configuración de replicación heredados del clon
for f in /etc/mysql/mariadb.conf.d/60-replication*.cnf; do
    [ -f "$f" ] && sudo mv "$f" "${f}.bak" && echo "Desactivado: $f"
done
# Si se hereda bind-address = 127.0.0.1 en 50-server.cnf, corregirlo
sudo sed -i \
    's/^bind-address[ ](:space:)*=.*\/bind-address = 0.0.0.0/' \
    /etc/mysql/mariadb.conf.d/50-server.cnf
# Crear el archivo de configuración del coordinador
sudo tee /etc/mysql/mariadb.conf.d/60-coordinador.cnf > /dev/null << 'EOF'
# =====
# Configuración del coordinador Spider — bdd-nodo06
# IP: 192.168.56.106 | server_id: 6
# Fase 14 del Laboratorio BDD.
# =====
[mariadb]
# Identificación única del nodo en el ecosistema
server_id          = 6
# El coordinador NO replica hacia otros nodos
# y NO actúa como esclavo de ningún maestro
log_bin            = OFF
skip_slave_start   = ON
# Sin restricción de solo lectura: el coordinador acepta escrituras
read_only          = OFF
# Aceptar conexiones desde cualquier interfaz de la red Host-Only
bind-address       = 0.0.0.0
# Requerido por Spider: evitar resolución DNS que cause timeouts
skip_name_resolve  = ON
EOF
```

Reiniciar MariaDB y verificar:

 Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb --no-pager -l
sudo mariadb << 'EOF'
SHOW VARIABLES LIKE 'server_id';
SHOW VARIABLES LIKE 'log_bin';
SHOW VARIABLES LIKE 'read_only';
SHOW VARIABLES LIKE 'bind_address';
SHOW VARIABLES LIKE 'skip_name_resolve';
EOF
```

Salida esperada:

 Text

```
server_id      → 6
log_bin        → OFF
read_only      → OFF
bind_address   → 0.0.0.0
skip_name_resolve → ON
```


E.10 Instalar y habilitar el motor Spider (VM — nodo06)

 Bash

```
sudo mariadb << 'EOF'

-- -----
-- Instalar el plugin Spider.
-- INSTALL SONAME carga ha_spider.so dinámicamente y crea las
-- tablas de sistema necesarias en la base de datos mysql.
-- En MariaDB 10.11 el plugin está incluido en el paquete base.
-- -----

INSTALL SONAME 'ha_spider';
-- Verificar que Spider quedó activo
SELECT PLUGIN_NAME,
       PLUGIN_VERSION,
       PLUGIN_STATUS,
       PLUGIN_TYPE
FROM information_schema.PLUGINS
WHERE PLUGIN_NAME = 'SPIDER';
-- Las tablas de sistema deben existir en mysql
SHOW TABLES IN mysql LIKE 'spider%';
-- Confirmar que Spider aparece como motor disponible
SELECT ENGINE, SUPPORT
FROM information_schema.ENGINES
WHERE ENGINE = 'SPIDER';
EOF
```

Salida esperada:

 Text

```
PLUGIN_NAME  PLUGIN_STATUS  PLUGIN_TYPE
SPIDER       ACTIVE          STORAGE ENGINE
Tablas spider* en mysql:
spider_link_failed_log  spider_link_mon_servers  spider_tables
spider_xa  spider_xa_failed_log  spider_xa_member
ENGINE    SUPPORT
SPIDER    YES
```

Si `PLUGIN_STATUS` es `DISABLED` o la instalación falla con `ERROR 1126` :

 Bash

```
# Instalar el paquete del plugin desde apt
sudo apt-get update && sudo apt-get install -y mariadb-plugin-spider
# Reintentar la instalación en MariaDB
sudo mariadb -e "INSTALL SONAME 'ha_spider';"
# Si las tablas de sistema no se crearon automáticamente:
SPIDER_SQL=$(find /usr/share/mysql* -name "install_spider.sql" 2>/dev/null |
head -1)
[ -n "$SPIDER_SQL" ] && sudo mariadb < "$SPIDER_SQL" && echo
"Script ejecutado."
```

E.11 Arrancar nodo04 y nodo05; verificar conectividad (host + VM)

 PowerShell

```
# Con nodo06 ya en .106, arrancar los shards sin conflicto de IP
VBoxManage startvm "bdd-nodo04" --type headless
VBoxManage startvm "bdd-nodo05" --type headless
Start-Sleep -Seconds 35
```

Verificar conectividad TCP desde nodo06 **antes** de crear tablas Spider:

 Bash

```
# En nodo06 - verificar acceso a nodo04
mysql -h 192.168.56.104 -P 3306 \
-u spider_user -p'Spider_2025!' \
-e "SELECT 'nodo04 accesible' AS estado, @@hostname AS host,
@@server_id AS srv_id;" 2>&1
# Verificar acceso a nodo05
mysql -h 192.168.56.105 -P 3306 \
-u spider_user -p'Spider_2025!' \
-e "SELECT 'nodo05 accesible' AS estado, @@hostname AS host,
@@server_id AS srv_id;" 2>&1
```

Salida esperada para cada nodo:

Text

```
+-----+-----+-----+
| estado      | host      | srv_id |
+-----+-----+-----+
| nodo04 accesible | bdd-nodo04 |      4 |
+-----+-----+-----+
```

Si alguno devuelve **ERROR 2003**, revisar la sección G antes de continuar.

E.12 Registrar los servidores remotos con `CREATE SERVER` (VM — nodo06)

 Bash

```
sudo mariadb << 'EOF'
-- =====
-- SERVIDORES REMOTOS
-- Las credenciales se almacenan una sola vez en mysql.servers.
-- Las tablas Spider referencian solo el nombre del servidor,
-- manteniendo el COMMENT limpio y evitando repetición.
-- =====
CREATE SERVER IF NOT EXISTS srv_nodo04
  FOREIGN DATA WRAPPER mysql
  OPTIONS (
    HOST      '192.168.56.104',
    PORT      3306,
    DATABASE  'lab_bdd',
    USER      'spider_user',
    PASSWORD  'Spider_2025!'
  );
CREATE SERVER IF NOT EXISTS srv_nodo05
  FOREIGN DATA WRAPPER mysql
  OPTIONS (
    HOST      '192.168.56.105',
    PORT      3306,
    DATABASE  'lab_bdd',
    USER      'spider_user',
    PASSWORD  'Spider_2025!'
  );
-- Verificar que ambos servidores quedaron registrados
SELECT Server_name AS servidor,
       Host         AS ip,
       Db           AS base_datos,
       Username     AS usuario,
       Port         AS puerto
FROM mysql.servers
ORDER BY Server_name;
EOF
```

Resultado esperado: 2 filas con `srv_nodo04` (`.104`) y `srv_nodo05` (`.105`).

E.13 Crear `lab_bdd` y tablas Spider para fragmentos horizontales (VM — nodo06)

 Bash

```
sudo mariadb << 'EOF'
-- Base de datos del coordinador
-- No contiene filas de datos de negocio; solo definiciones Spider y VIEWS
CREATE DATABASE IF NOT EXISTS lab_bdd
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci
    COMMENT 'Coordinador Spider — Fase 14. Metadatos sin datos locales.';
USE lab_bdd;
-- =====
-- TABLA SPIDER: clientes
-- PARTITION BY LIST COLUMNS (region):
--   frag_A (norte, este) → nodo04
--   frag_B (sur, oeste) → nodo05
-- Con WHERE region = 'norte', Spider envía la subconsulta solo a
-- nodo04 (poda remota activa). Sin filtro, consulta ambos y hace
-- UNION ALL de los resultados.
-- PRIMARY KEY (id, region) es obligatoria con PARTITION Spider.
-- =====
CREATE TABLE IF NOT EXISTS clientes (
    id                INT                NOT NULL AUTO_INCREMENT,
    nombre            VARCHAR(100)       NOT NULL,
    apellido          VARCHAR(100)       NOT NULL,
    email             VARCHAR(150),
    telefono          VARCHAR(20),
    region            ENUM('norte', 'sur', 'este', 'oeste') NOT NULL,
    ciudad            VARCHAR(100),
    fecha_alta        DATETIME           DEFAULT CURRENT_TIMESTAMP,
    ultima_modificacion DATETIME         DEFAULT NULL,
    PRIMARY KEY (id, region)
) ENGINE = SPIDER
PARTITION BY LIST COLUMNS (region) (
    PARTITION frag_A VALUES IN ('norte', 'este')
        COMMENT = 'server "srv_nodo04", table "clientes"',
    PARTITION frag_B VALUES IN ('sur', 'oeste')
        COMMENT = 'server "srv_nodo05", table "clientes"'
);
-- =====
-- TABLA SPIDER: pedidos
-- Mismo esquema de particionamiento que clientes.
```

```

-- =====
CREATE TABLE IF NOT EXISTS pedidos (
    id            INT            NOT NULL AUTO_INCREMENT,
    cliente_id    INT            NOT NULL,
    region        ENUM('norte','sur','este','oeste') NOT NULL,
    fecha_pedido  DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado        ENUM('pendiente','procesado','enviado',
                      'entregado','cancelado') DEFAULT 'pendiente',
    total         DECIMAL(10,2),
    PRIMARY KEY (id, region)
) ENGINE = SPIDER
PARTITION BY LIST COLUMNS (region) (
    PARTITION frag_A VALUES IN ('norte', 'este')
        COMMENT = 'server "srv_nodo04", table "pedidos"',
    PARTITION frag_B VALUES IN ('sur', 'oeste')
        COMMENT = 'server "srv_nodo05", table "pedidos"'
);
-- =====
-- TABLAS SPIDER para detalle_pedidos
-- detalle_pedidos no tiene columna region: no se puede aplicar
-- PARTITION BY LIST COLUMNS (region) directamente.
-- Solución: tabla Spider simple por shard + VIEW UNION ALL.
-- =====
CREATE TABLE IF NOT EXISTS spider_detalle_nodo04 (
    id            INT            NOT NULL AUTO_INCREMENT,
    pedido_id     INT            NOT NULL,
    producto_id   INT            NOT NULL,
    cantidad      INT            NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal      DECIMAL(10,2),
    PRIMARY KEY (id)
) ENGINE = SPIDER
COMMENT = 'server "srv_nodo04", table "detalle_pedidos"';
CREATE TABLE IF NOT EXISTS spider_detalle_nodo05 (
    id            INT            NOT NULL AUTO_INCREMENT,
    pedido_id     INT            NOT NULL,
    producto_id   INT            NOT NULL,
    cantidad      INT            NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal      DECIMAL(10,2),
    PRIMARY KEY (id)
) ENGINE = SPIDER
COMMENT = 'server "srv_nodo05", table "detalle_pedidos"';
-- VIEW que unifica ambos shards de detalle_pedidos
CREATE OR REPLACE VIEW detalle_pedidos AS

```

```

SELECT * FROM spider_detalle_nodo04
UNION ALL
SELECT * FROM spider_detalle_nodo05;

-- =====
-- VERIFICACIÓN INMEDIATA - fragmentos horizontales desde nodo06
-- =====
SELECT 'clientes (Spider, 2 shards)'      AS fuente, COUNT(*) AS filas
FROM clientes
UNION ALL
SELECT 'pedidos (Spider, 2 shards)',      COUNT(*)
FROM pedidos
UNION ALL
SELECT 'detalle_pedidos (VIEW UNION ALL)', COUNT(*)
FROM detalle_pedidos;
-- Resultado esperado: 20 / 20 / ~36
EOF

```

E.14 Crear tablas Spider para fragmentos verticales y VIEW productos (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- TABLA SPIDER: v_productos_basico → nodo04
-- Columnas operacionales. Solo activa nodo04 cuando se consulta.
-- =====
CREATE TABLE IF NOT EXISTS v_productos_basico (
    id            INT            NOT NULL AUTO_INCREMENT,
    sku           VARCHAR(50)    NOT NULL,
    nombre        VARCHAR(200)   NOT NULL,
    categoria     VARCHAR(100),
    precio        DECIMAL(10,2)  NOT NULL,
    stock         INT            DEFAULT 0,
    fecha_creacion DATETIME      DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_sku (sku)
) ENGINE = SPIDER
  COMMENT = 'server "srv_nodo04", table "v_productos_basico"';

-- =====
-- TABLA SPIDER: v_productos_detalle → nodo05
-- Columnas de detalle (TEXT). Solo activa nodo05 cuando se consulta.
-- =====
CREATE TABLE IF NOT EXISTS v_productos_detalle (
    id            INT            NOT NULL,
    sku           VARCHAR(50)    NOT NULL,
    descripcion   TEXT,
    ficha_tecnica TEXT,
    imagen_url    VARCHAR(500),
    peso_kg       DECIMAL(8,3),
    PRIMARY KEY (id),
    KEY idx_sku (sku)
) ENGINE = SPIDER
  COMMENT = 'server "srv_nodo05", table "v_productos_detalle"';

-- =====
-- VIEW: productos
-- Reconstruye la relación completa mediante JOIN entre los dos
-- fragmentos verticales. Para el cliente en nodo06:
--   SELECT * FROM productos → indistinguible de una tabla local.
-- Internamente: Spider envía subconsulta a nodo04 y a nodo05,
-- el coordinador hace el JOIN y devuelve la fila completa.
```



```

-- =====
CREATE OR REPLACE VIEW productos AS
SELECT
    b.id,
    b.sku,
    b.nombre,
    b.categoria,
    b.precio,
    b.stock,
    d.descripcion,
    d.ficha_tecnica,
    d.imagen_url,
    d.peso_kg,
    b.fecha_creacion
FROM v_productos_basico b
JOIN v_productos_detalle d ON b.id = d.id;
-- =====
-- VERIFICACIONES DE LOS FRAGMENTOS VERTICALES
-- =====
SELECT 'v_productos_basico (Spider → nodo04)' AS fuente, COUNT(*)
AS productos
FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle (Spider → nodo05)', COUNT(*)
FROM v_productos_detalle
UNION ALL
SELECT 'productos VIEW (JOIN distribuido)', COUNT(*)
FROM productos;
-- Resultado esperado: 10 / 10 / 10
-- Inspección del primer producto completamente reconstruido
SELECT id, sku, nombre, categoria, precio, stock,
    LEFT(descripcion, 50) AS descripcion_preview,
    LEFT(ficha_tecnica, 50) AS ficha_preview,
    imagen_url,
    peso_kg
FROM productos
WHERE id = 1;
EOF

```

E.15 Consultas de verificación distribuida desde nodo06 (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- CONSULTA 1: Clientes de una sola región – poda Spider activa
-- WHERE region = 'norte' → Spider consulta solo frag_A (nodo04).
-- =====
SELECT id,
       CONCAT(nombre, ' ', apellido) AS cliente,
       region,
       ciudad
FROM   clientes
WHERE  region = 'norte'
ORDER BY id;
-- =====
-- CONSULTA 2: Conteo de clientes por región (ambos shards)
-- Sin filtro → Spider consulta nodo04 Y nodo05 con UNION ALL.
-- =====
SELECT region,
       COUNT(*) AS clientes_por_region
FROM   clientes
GROUP BY region
ORDER BY region;
-- =====
-- CONSULTA 3: Productos con stock disponible (solo V_basico)
-- Solo activa nodo04; nodo05 no interviene.
-- =====
SELECT sku, nombre, categoria, precio, stock
FROM   v_productos_basico
WHERE  stock > 0
ORDER BY precio DESC;
-- =====
-- CONSULTA 4: Ficha completa de un producto (JOIN distribuido)
-- Activa nodo04 (v_productos_basico) Y nodo05 (v_productos_detalle).
-- =====
SELECT id, sku, nombre, categoria, precio, stock,
       descripcion, ficha_tecnica, imagen_url, peso_kg
FROM   productos
WHERE  id = 2;
-- =====
-- CONSULTA 5: Pedidos con datos de cliente (JOIN en shards)
-- Spider usa poda en ambas tablas particionadas.
-- =====
```

```

SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       p.id                               AS pedido,
       p.estado,
       p.total
FROM   clientes c
JOIN   pedidos p ON p.cliente_id = c.id
ORDER BY c.region, p.id;

-- =====
-- CONSULTA 6: HÍBRIDA – facturación por región con categorías
-- Cruza fragmentación HORIZONTAL (clientes/pedidos/detalle)
-- con fragmentación VERTICAL (v_productos_basico para categoría).
-- =====
SELECT c.region,
       COUNT(DISTINCT c.id)                AS clientes_activos,
       COUNT(DISTINCT p.id)                AS num_pedidos,
       GROUP_CONCAT(DISTINCT vb.categoria
                    ORDER BY vb.categoria) AS categorias,
       ROUND(SUM(dp.subtotal), 2)          AS facturacion_total
FROM   clientes c
JOIN   pedidos p ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id = dp.producto_id
GROUP BY c.region
ORDER BY facturacion_total DESC;

-- Resultado esperado: 4 filas con datos de facturación no nulos
-- =====
-- CONSULTA 7: EXPLAIN – demostración de poda Spider
-- =====
EXPLAIN SELECT id, nombre, ciudad
FROM   clientes
WHERE  region = 'norte';
EXPLAIN SELECT id, nombre, ciudad
FROM   clientes;

-- =====
-- RESUMEN GLOBAL – todas las métricas desde el coordinador
-- =====
SELECT 'clientes (2 shards, PARTITION Spider)' AS metrica, COUNT(*) AS
valor FROM clientes
UNION ALL
SELECT 'pedidos (2 shards, PARTITION Spider)', COUNT(*)
FROM pedidos
UNION ALL
SELECT 'detalle_pedidos (VIEW UNION ALL)', COUNT(*)
FROM detalle_pedidos

```

```

UNION ALL
SELECT 'v_productos_basico (Spider → nodo04)',          COUNT(*)
FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle (Spider → nodo05)',          COUNT(*)
FROM v_productos_detalle
UNION ALL
SELECT 'productos VIEW (JOIN distribuido)',              COUNT(*)
FROM productos;
-- Resultado esperado:
-- clientes                → 20
-- pedidos                 → 20
-- detalle_pedidos         → ~36
-- v_productos_basico     → 10
-- v_productos_detalle    → 10
-- productos               → 10
EOF

```

E.16 Apagar las VMs y tomar los snapshots fase14-completa (host)

 Bash

```

# Desde la sesión SSH de nodo06
sudo poweroff
# Desde la sesión SSH de nodo04 (si sigue abierta)
sudo poweroff
# Desde la sesión SSH de nodo05 (si sigue abierta)
sudo poweroff

```

Confirmar:

 PowerShell

```

VBoxManage list runningvms
# Salida esperada: vacía

```

Tomar los snapshots en los seis nodos:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase14-completa" `
--description "Sin cambios en Fase 14. Nodo de referencia. Snapshot de hito."
VBoxManage snapshot "bdd-nodo02" take "fase14-completa" `
--description "Sin cambios en Fase 14. Esclavo de replicacion. Snapshot
de hito."
VBoxManage snapshot "bdd-nodo03" take "fase14-completa" `
--description "Sin cambios en Fase 14. Nodo multi-maestro. Snapshot de hito."
VBoxManage snapshot "bdd-nodo04" take "fase14-completa" `
--description "SHARD-A: v_productos_basico creada y poblada (10 filas).
spider_user@192.168.56.106 con SELECT/INSERT/UPDATE/DELETE en lab_bdd.*."
VBoxManage snapshot "bdd-nodo05" take "fase14-completa" `
--description "SHARD-B: v_productos_detalle creada y poblada (10 filas).
spider_user@192.168.56.106 con SELECT/INSERT/UPDATE/DELETE en lab_bdd.*."
VBoxManage snapshot "bdd-nodo06" take "fase14-completa" `
--description "COORDINADOR: Spider ACTIVE, server_id=6, log_bin=OFF. CREATE
SERVER nodo04/nodo05. Spider tables: clientes, pedidos (PARTITION LIST
region), spider_detalle_nodo04/05, v_productos_basico, v_productos_detalle.
VIEWS: detalle_pedidos (UNION ALL), productos (JOIN). Consultas
distribuidas verificadas."
```

Confirmar:

 PowerShell

```
VBoxManage snapshot "bdd-nodo04" list
VBoxManage snapshot "bdd-nodo05" list
VBoxManage snapshot "bdd-nodo06" list
```

nodo04 y nodo05 deben mostrar `fase05-completa` hasta `fase14-completa`. nodo06 muestra únicamente `fase14-completa` (primer snapshot de esta VM).

F. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes:

1. `SELECT COUNT(*) FROM lab_bdd.v_productos_basico` en **nodo04** devuelve exactamente **10**.

2. `SELECT COUNT(*) FROM lab_bdd.v_productos_detalle` en **nodo05** devuelve exactamente **10**.
3. `DESCRIBE lab_bdd.v_productos_basico` en **nodo04** **no muestra** las columnas `descripcion`, `ficha_tecnica`, `imagen_url` ni `peso_kg`.
4. `DESCRIBE lab_bdd.v_productos_detalle` en **nodo05** **no muestra** las columnas `nombre`, `categoria`, `precio`, `stock` ni `fecha_creacion`.
5. Las verificaciones de correctitud del paso E.5 devuelven **OK** para completitud y disjunción de columnas en ambos nodos.
6. `ip addr show` en **nodo06** muestra la dirección `192.168.56.106/24`.
7. `hostname` en **nodo06** devuelve `bdd-nodo06`.
8. `SHOW VARIABLES LIKE 'server_id'` en **nodo06** devuelve **6**.
9. `SHOW VARIABLES LIKE 'log_bin'` en **nodo06** devuelve **OFF**.
10. `SELECT PLUGIN_STATUS FROM information_schema.PLUGINS WHERE PLUGIN_NAME = 'SPIDER'` en **nodo06** devuelve **ACTIVE**.
11. `SHOW TABLES IN mysql LIKE 'spider%'` en **nodo06** devuelve al menos **6 tablas** de sistema.
12. `SELECT Server_name, Host FROM mysql.servers ORDER BY Server_name` en **nodo06** muestra exactamente **2 filas**: `srv_nodo04` con IP `.104` y `srv_nodo05` con IP `.105`.
13. `mysql -h 192.168.56.104 -u spider_user -p'Spider_2025!' -e "SELECT 1"` ejecutado **desde nodo06** devuelve **1** sin errores.
14. `mysql -h 192.168.56.105 -u spider_user -p'Spider_2025!' -e "SELECT 1"` ejecutado **desde nodo06** devuelve **1** sin errores.
15. `SELECT COUNT(*) FROM lab_bdd.clientes` en **nodo06** devuelve **20**.
16. `SELECT COUNT(*) FROM lab_bdd.pedidos` en **nodo06** devuelve **20**.
17. `SELECT COUNT(*) FROM lab_bdd.detalle_pedidos` en **nodo06** devuelve el total correcto.
18. `SELECT COUNT(*) FROM lab_bdd.v_productos_basico` en **nodo06** devuelve **10** (desde **nodo04**).
19. `SELECT COUNT(*) FROM lab_bdd.v_productos_detalle` en **nodo06** devuelve **10** (desde **nodo05**).
20. `SELECT COUNT(*) FROM lab_bdd.productos` (VIEW) en **nodo06** devuelve **10** (JOIN distribuido).
21. La Consulta 6 (híbrida) de E.15 devuelve exactamente **4 filas** con valores de facturación no nulos, una por cada región del dominio.

22. `EXPLAIN SELECT * FROM clientes WHERE region = 'norte'` en nodo06 muestra únicamente `frag_A` en la columna `partitions`.
 23. `EXPLAIN SELECT * FROM clientes` sin filtro en nodo06 muestra **ambas** particiones (`frag_A` y `frag_B`).
 24. `SHOW TABLES IN lab_bdd` en nodo06 devuelve exactamente: `clientes`, `pedidos`, `spider_detalle_nodo04`, `spider_detalle_nodo05`, `v_productos_basico`, `v_productos_detalle`, y las VIEWS `detalle_pedidos` y `productos`.
 25. Los snapshots `fase14-completa` existen en los **seis nodos** del laboratorio.
-

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>INSTALL SONAME</code> <code>'ha_spider'</code> falla con <code>ERROR</code> <code>1126: Can't</code> <code>open shared</code> <code>library</code> <code>'ha_spider'</code>	El archivo <code>ha_spider.so</code> no está en la instalación actual de MariaDB	Ejecutar <code>sudo</code> <code>apt-get</code> <code>install -y</code> <code>mariadb-</code> <code>plugin-spider</code> y reintentar el <code>INSTALL</code> <code>SONAME</code> ; verificar ruta con <code>find</code> <code>/usr/lib/mysql</code> <code>/usr/lib/maria</code> <code>db -name</code> <code>"ha_spider"</code> <code>2>/dev/null</code>
<code>SELECT ...</code> <code>FROM clientes</code> en nodo06 devuelve <code>ERROR</code> <code>12502: Unable</code> <code>to connect to</code> <code>foreign data</code> <code>source</code>	Spider no alcanza el shard remoto: <code>bind-</code> <code>address =</code> <code>127.0.0.1</code> en nodo04/05, firewall activo, o credenciales incorrectas	Probar desde nodo06: <code>mysql</code> <code>-h</code> <code>192.168.56.104</code> <code>-u spider_user</code> <code>-</code> <code>p'Spider_2025!</code> <code>' -e "SELECT</code> <code>1" ; si falla,</code> verificar <code>SHOW</code> <code>VARIABLES LIKE</code> <code>'bind_address</code> <code>'</code> en nodo04 y que el usuario existe con <code>SELECT User,</code> <code>Host FROM</code> <code>mysql.user</code> <code>WHERE</code> <code>User='spider_u</code> <code>ser'</code>
<code>ERROR 1429:</code> <code>Unable to</code> <code>connect to</code>	Las credenciales de <code>CREATE SERVER</code>	Verificar en nodo04/05: <code>SHOW GRANTS</code>

<code>foreign data</code> <code>source</code> al crear tabla Spider	no coinciden con el usuario real del nodo remoto	<pre>FOR 'spider_user'@ '192.168.56.10 6'; ejecutar GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.* TO 'spider_user'@ '192.168.56.10 6'; FLUSH PRIVILEGES; si el usuario no tiene los privilegios correctos</pre>
La VIEW <code>productos</code> devuelve 0 filas aunque los fragmentos tienen 10 filas cada uno	Los IDs en <code>v_productos_ba</code> <code>sico</code> (nodo04) y en <code>v_productos_de</code> <code>talle</code> (nodo05) no coinciden	Verificar con <code>SELECT id FROM</code> <code>lab_bdd.v_prod</code> <code>uctos_basico</code> <code>ORDER BY id</code> en nodo04 y <code>SELECT id FROM</code> <code>lab_bdd.v_prod</code> <code>uctos_detalle</code> <code>ORDER BY id</code> en nodo05; si difieren, repoblar el fragmento incorrecto con <code>TRUNCATE</code> <code>TABLE; INSERT</code> <code>INTO ...</code> <code>SELECT ...</code> <code>FROM productos</code> <code>ORDER BY id</code>
<code>CREATE SERVER</code> falla con <code>ERROR</code> <code>1409: Failed</code> <code>to create</code>	Ya existe un servidor con ese nombre de una ejecución previa	Ejecutar <code>DROP</code> <code>SERVER IF</code> <code>EXISTS</code> <code>srv_nodo04;</code> <code>DROP SERVER IF</code>

<pre>default foreign server</pre>		<pre>EXISTS srv_nodo05; y recrearlos</pre>
La tabla Spider <code>clientes</code> en nodo06 devuelve error por columna <code>ultima_modificacion</code>	La definición Spider incluye esa columna pero la tabla remota fue creada antes de que existiera (snapshot pre-Fase 10)	Verificar con <code>DESCRIBE lab_bdd.clientes</code> en nodo04; si la columna no existe, eliminarla de la definición Spider: <code>ALTER TABLE clientes DROP COLUMN ultima_modificacion;</code> en nodo06
<pre>VBoxManage clonevm falla con Could not find a snapshot named 'fase07- completa'</pre>	El nombre exacto del snapshot difiere (espacios, mayúsculas, caracteres especiales)	Ejecutar <code>VBoxManage snapshot "bdd-nodo01" list</code> y copiar el nombre exacto del snapshot correspondiente a la instalación de MariaDB
SSH a <code>.101</code> conecta al nodo equivocado durante la configuración de nodo06	nodo01 sigue encendido con la misma IP que el clon	Verificar con <code>VBoxManage list runningvms</code> que nodo01 esté apagado antes de arrancar nodo06 (paso E.6)
<pre>netplan apply</pre> falla con error de sintaxis YAML	El archivo <code>.yaml</code> tiene problemas de indentación	Usar el comando <code>sed -i 's/192\\.168\\.56\\.101/192.168.56.106/g'</code> en lugar de

		editar manualmente; validar con <code>sudo netplan try</code> antes de aplicar; YAML requiere espacios, no tabulaciones
<code>EXPLAIN</code> en tabla Spider particionada no muestra la columna <code>partitions</code>	Algunos clientes MariaDB no despliegan esta columna automáticamente e para tablas Spider	Usar la sintaxis alternativa <code>EXPLAIN PARTITIONS SELECT ...;</code> también se puede verificar la poda observando <code>SHOW STATUS LIKE 'Handler_read%</code> antes y después de consultas con y sin filtro de región
La Consulta 6 (híbrida) devuelve menos de 4 filas	Alguna región no tiene líneas de <code>detalle_pedido</code>s con coincidencia en <code>v_productos_basico</code>	Verificar que <code>spider_detalle_nodo04</code> y <code>spider_detalle_nodo05</code> tienen filas (<code>SELECT COUNT(*) FROM spider_detalle_nodo04</code> en nodo06); cambiar el <code>JOIN v_productos_basico</code> por <code>LEFT JOIN</code> si el problema es exclusión de

		regiones sin productos comprados
<pre>SHOW TABLES IN mysql LIKE 'spider%' devuelve vacío tras INSTALL SONAME</pre>	Las tablas de sistema de Spider no se crearon automáticamente	Localizar el script con <code>find</code> <pre>/usr/share/mys ql* -name "install_spide r.sql" 2>/dev/null y ejecutar con sudo mariadb < ruta_del_scrip t.sql</pre>
El snapshot <code>fase14-completa</code> de nodo06 falla porque la VM está en ejecución	La VM no se apagó correctamente antes de tomar el snapshot	Confirmar con <code>VBoxManage</code> <pre>list runningvms ; si aparece, apagar con VBoxManage controlvm "bdd-nodo06" acpipowerbutto n y esperar 20 segundos antes de reintentar</pre>

H. Checklist de validación

- ☐ `spider_user@'192.168.56.106'` existe con `SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*` en **nodo04** y en **nodo05** (verificado con `SHOW GRANTS`).
- ☐ `v_productos_basico` existe en nodo04 con exactamente **10 filas** y columnas `id, sku, nombre, categoria, precio, stock, fecha_creacion`. Sin columnas TEXT de detalle.
- ☐ `v_productos_detalle` existe en nodo05 con exactamente **10 filas** y columnas `id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg`. Sin columnas operacionales.

- ☐ Las verificaciones de correctitud del paso E.5 devuelven **OK** (completitud y disjunción de columnas) en ambos nodos.
- ☐ `bdd-nodo06` responde en `192.168.56.106` (verificado con `ip addr show`).
- ☐ `hostname` en nodo06 devuelve `bdd-nodo06`.
- ☐ `SHOW VARIABLES LIKE 'server_id'` devuelve **6** en nodo06.
- ☐ `SHOW VARIABLES LIKE 'log_bin'` devuelve **OFF** en nodo06.
- ☐ `SHOW VARIABLES LIKE 'read_only'` devuelve **OFF** en nodo06.
- ☐ `SELECT PLUGIN_STATUS FROM information_schema.PLUGINS WHERE PLUGIN_NAME = 'SPIDER'` devuelve **ACTIVE** en nodo06.
- ☐ `SHOW TABLES IN mysql LIKE 'spider%'` devuelve al menos **6 tablas** de sistema en nodo06.
- ☐ `SELECT * FROM mysql.servers` muestra exactamente **2 filas** en nodo06:
`srv_nodo04` (IP `.104`) y `srv_nodo05` (IP `.105`).
- ☐ Conectividad verificada: `mysql -h 192.168.56.104 -u spider_user` desde nodo06 retorna `1`.
- ☐ Conectividad verificada: `mysql -h 192.168.56.105 -u spider_user` desde nodo06 retorna `1`.
- ☐ `SELECT COUNT(*) FROM lab_bdd.clientes` en nodo06 devuelve **20**.
- ☐ `SELECT COUNT(*) FROM lab_bdd.pedidos` en nodo06 devuelve **20**.
- ☐ `SELECT COUNT(*) FROM lab_bdd.detalle_pedidos` en nodo06 devuelve el total correcto.
- ☐ `SELECT COUNT(*) FROM lab_bdd.v_productos_basico` en nodo06 devuelve **10**.
- ☐ `SELECT COUNT(*) FROM lab_bdd.v_productos_detalle` en nodo06 devuelve **10**.
- ☐ `SELECT COUNT(*) FROM lab_bdd.productos` (VIEW) en nodo06 devuelve **10**.
- ☐ La Consulta 6 (híbrida, E.15) devuelve **4 filas** con facturación no nula.
- ☐ `EXPLAIN SELECT * FROM clientes WHERE region = 'norte'` en nodo06 muestra solo `frag_A`.
- ☐ `EXPLAIN SELECT * FROM clientes` sin filtro muestra `frag_A` y `frag_B` en nodo06.
- ☐ `SHOW TABLES IN lab_bdd` en nodo06 devuelve exactamente 6 tablas + 2 VIEWS.
- ☐ Los snapshots `fase14-completa` existen en los **6 nodos** del laboratorio.
- ☐ Puedo explicar sin ver el documento por qué `detalle_pedidos` usa dos tablas Spider simples + VIEW en lugar de una tabla Spider particionada por `region`.
- ☐ Puedo describir el recorrido completo de la consulta `SELECT * FROM productos WHERE id = 5` desde nodo06: qué subconsultas genera Spider, a qué nodos las envía y cómo combina los resultados.

- ☐ Puedo distinguir la transparencia de fragmentación (ocultar la división de datos) de la transparencia de ubicación (ocultar en qué nodo físico están los datos).
-

Preguntas teóricas para estudiantes

1. La VIEW `productos` en nodo06 realiza un `JOIN` entre `v_productos_basico` (Spider → nodo04) y `v_productos_detalle` (Spider → nodo05). Analiza qué ocurre internamente en el coordinador cuando el cliente ejecuta `SELECT nombre, descripcion FROM productos WHERE id = 3`: ¿Spider puede empujar el predicado `WHERE id = 3` hacia ambos nodos antes de traer los datos (*predicate pushdown*), o primero trae todas las filas de cada fragmento y luego filtra localmente? ¿Qué volumen de datos cruzaría la red en cada caso para una tabla de un millón de productos? Relaciona la respuesta con el concepto de eficiencia de JOIN distribuido.
2. La tabla Spider `clientes` en nodo06 tiene `PRIMARY KEY (id, region)`, mientras que la tabla remota en nodo04 tiene `PRIMARY KEY (id)`. Explica por qué Spider permite esta discrepancia de definición de clave primaria. Luego analiza: si un cliente ejecuta `INSERT INTO clientes (id, nombre, apellido, region, ...) VALUES (21, 'Ana', 'López', 'norte', ...)` directamente en nodo06, ¿ese INSERT llega a nodo04?, ¿qué validaciones realiza Spider antes de reenviar la operación al shard remoto?, ¿qué error recibirá si intenta insertar un cliente con `region = 'centro'` (valor no definido en las particiones)?
3. `detalle_pedidos` en nodo06 se implementó con dos tablas Spider simples más una `VIEW UNION ALL`. Un estudiante propone agregar la columna `region` directamente a `detalle_pedidos` en nodo04 y nodo05 (desnormalización) para poder usar `PARTITION BY LIST COLUMNS (region)` igual que `clientes` y `pedidos`. Evalúa esta propuesta: ¿qué DDL habría que ejecutar en los shards?, ¿qué mecanismo garantizaría que `detalle_pedidos.region` coincida siempre con `pedidos.region` sin FK de motor?, y ¿qué gana el sistema en términos de rendimiento de poda si se adopta esta solución?
4. `CREATE SERVER` almacena la contraseña de `spider_user` en `mysql.servers`, visible para cualquier usuario con acceso `SELECT` a esa tabla en nodo06. Describe el vector de ataque concreto si un atacante obtiene acceso de solo lectura a `mysql.servers`, e identifica tres medidas de mitigación distintas —sin cambiar la arquitectura Spider— que reducirían ese riesgo. Una de ellas debe involucrar los privilegios concedidos a `spider_user` en nodo04 y nodo05; otra debe ser a nivel del sistema operativo de nodo06.

5. Clasifica los seis nodos del laboratorio según el teorema CAP, justificando cada clasificación con argumentos técnicos concretos del diseño actual. Por ejemplo: ante una partición de red entre nodo06 y nodo05 mientras una consulta a la VIEW `productos` está en curso, ¿el coordinador elige Consistencia (rechaza la consulta) o Disponibilidad (devuelve datos parciales de nodo04)? ¿Y el par nodo01-nodo02 en replicación asíncrona ante la caída de nodo01 antes de que un evento del binlog llegara a nodo02?
-

Ejercicios prácticos

1. Monitoreo del tráfico Spider entre coordinador y shards.

En nodo04, ejecutar `sudo tcpdump -i enp0s8 -n port 3306 -c 100` en segundo plano mientras

se ejecutan desde nodo06 las siguientes tres consultas en secuencia:

- (a) `SELECT precio FROM v_productos_basico WHERE id = 1`,
- (b) `SELECT descripcion FROM v_productos_detalle WHERE id = 1`,
- © `SELECT * FROM productos WHERE id = 1` (VIEW).

Repetir el experimento con el tcpdump en nodo05. Para cada consulta, registrar en qué nodo(s) aparece tráfico TCP. Concluir: ¿la consulta (a) activa solo nodo04?, ¿la consulta (b) activa solo nodo05?, ¿la VIEW de © activa ambos nodos incluso cuando el `WHERE` filtra una sola fila por `id`? Documentar la salida del tcpdump con evidencia.

2. Inserción coordinada de un nuevo producto y observación del estado intermedio.

Insertar manualmente el producto `id = 11` en dos pasos separados:

- (a) Ejecutar `INSERT INTO lab_bdd.v_productos_basico (id, sku, nombre, ...)` directamente

en nodo04. Luego consultar `SELECT COUNT(*) FROM productos` desde nodo06 y observar qué devuelve la VIEW en estado intermedio (solo un fragmento insertado).

- (b) Ejecutar `INSERT INTO lab_bdd.v_productos_detalle (id, sku, descripcion, ...)` en nodo05.

Consultar nuevamente `SELECT * FROM productos WHERE id = 11` desde nodo06 y verificar que ahora devuelve la fila completa.

© Eliminar el producto de ambos fragmentos y confirmar que la VIEW vuelve a 10 filas.

Discutir: ¿qué problema de consistencia expone el estado intermedio? ¿Cómo lo resolvería una transacción distribuida XA en un entorno de producción real?

3. Script de auditoría de sincronía entre fragmentos verticales.

Escribir un script Bash en nodo06 que conecte a nodo04 y a nodo05 con `mysql -h IP -u spider_user` y compare los conjuntos de `id` presentes en `v_productos_basico` y

en `v_productos_detalle` . El script debe imprimir `[OK] Fragmentos sincronizados:`
`N productos` si los IDs coinciden, o `[FALLA] IDs solo en V_basico: X / solo en`
`V_detalle: Y` si difieren. Ejecutarlo en condiciones normales, luego insertar
deliberadamente una fila solo en nodo04 para crear un desajuste, confirmar que el
script lo detecta, eliminar la fila y verificar que el script vuelve a reportar `OK` .

Reto adicional para alumnos avanzados

Investigar y diseñar una solución para **escritura distribuida transparente desde nodo06**
a través de la VIEW `productos` . La VIEW actual es de solo lectura porque MariaDB no sabe
cómo descomponer automáticamente un `INSERT INTO productos (...)` en dos inserciones
separadas hacia dos tablas Spider en dos nodos distintos.

Implementar una de las dos soluciones siguientes y documentar el resultado completo:

Opción A — Stored Procedure:

Crear `CALL insertar_producto(sku, nombre, categoria, precio, stock, descripcion,`
`ficha_tecnica,`
`imagen_url, peso_kg)` en nodo06 que: (1) inserta las columnas operacionales en nodo04 via
la tabla Spider `v_productos_basico` , captura el `LAST_INSERT_ID()` generado, y (2) inserta
las columnas de detalle en nodo05 via `v_productos_detalle` usando ese mismo ID.

Demostrar

la llamada, verificar que el nuevo producto aparece en la VIEW `productos` , y discutir qué
garantía de atomicidad ofrece este procedimiento ante la caída de nodo05 entre los pasos 1
y 2.

Opción B — Trigger sobre VIEW:

Investigar si MariaDB soporta triggers `INSTEAD OF` sobre VIEWS (disponibles en SQLite y
PostgreSQL, no en MySQL/MariaDB estándar). Si no es posible, demostrar ese límite con
el error que produce intentar crear el trigger, explicar técnicamente por qué el motor no
puede soportarlo sin cambios en la arquitectura, y proponer qué extensión de Spider o qué
configuración de replicación permitiría escrituras bidireccionales transparentes desde nodo06.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Fragmentos verticales en nodo04/nodo05	20%	<code>v_productos_ba</code> <code>sico</code> y <code>v_productos_de</code> <code>talle</code> existen con las columnas correctas y 10 filas; las verificaciones de completitud y disjunción devuelven <code>OK</code>; el estudiante relaciona las columnas con el criterio de afinidad del DDD (Fase 9)
Configuración de nodo06 (MariaDB + Spider)	20%	nodo06 tiene IP <code>.106</code>, <code>server_id = 6</code>, <code>log_bin = OFF</code>; Spider <code>ACTIVE</code>; tablas <code>spider_*</code> presentes; el estudiante puede mostrar <code>SHOW VARIABLES</code> con los valores esperados y explicar por qué <code>log_bin = OFF</code> en el coordinador
<code>CREATE SERVER</code> y conectividad Spider	15%	<code>mysql.servers</code> muestra los dos servidores; la

		prueba de conectividad TCP desde nodo06 pasa sin error; el estudiante puede explicar la diferencia entre usar <code>CREATE SERVER</code> vs. credenciales directas en el <code>COMMENT</code> y las implicaciones de seguridad de cada opción
Tablas Spider horizontales y poda	20%	<code>clientes</code> y <code>pedidos</code> en nodo06 devuelven 20 filas; <code>EXPLAIN</code> confirma poda con filtro de región; el estudiante justifica por qué <code>detalle_pedido</code>s requiere un diseño diferente y lo demuestra con la implementación de dos tablas Spider + VIEW
Fragmentos verticales via Spider y VIEW productos	15%	<code>v_productos_basico</code> y <code>v_productos_detalle</code> en nodo06 devuelven 10 filas; VIEW <code>productos</code> devuelve 10 filas con todos los atributos; el

		estudiante puede describir el JOIN distribuido que ocurre internamente en Spider para resolver esa VIEW
Consultas distribuidas y transparencia	10%	Las 6 consultas de E.15 producen resultados correctos; la Consulta 6 (híbrida) devuelve 4 filas; el estudiante puede explicar para cada consulta cuáles nodos activa Spider, cuándo aplica poda y cuándo realiza UNION ALL

Preparación para la siguiente fase

La **Fase 15: Fragmentación Híbrida** requerirá:

- `bdd-nodo06` completamente operativo con Spider (esta fase): todas las tablas Spider y VIEWS funcionando, conectividad verificada hacia nodo04 y nodo05, snapshots `fase14-completa` en los seis nodos.
- Comprensión clara de la diferencia de rendimiento entre acceso a un solo fragmento (`v_productos_basico`, solo nodo04) versus JOIN distribuido (VIEW `productos`, nodo04
 - nodo05), establecida experimentalmente en las consultas de E.15.
- El Documento de Diseño Distribuido (`C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md`, sección D.5, Consulta Distribuida 4) como referencia técnica: esa consulta híbrida es exactamente la Consulta 6 de esta fase, ahora ejecutándose sobre la infraestructura real.

En la Fase 15 se analizarán en profundidad las consultas que demuestran la **fragmentación híbrida** como propiedad emergente del diseño actual: el cliente conectado a nodo06 ejecutará SQL que accede simultáneamente a fragmentos horizontales (clientes y pedidos por región entre nodo04 y nodo05) y a fragmentos verticales (columnas de productos entre nodo04 y nodo05), todo coordinado por Spider sin que el cliente perciba la distribución. Se introducirán además estrategias de optimización: cuándo conviene filtrar antes del JOIN, cómo reducir el volumen de datos transferidos entre coordinador y shards, y qué índices en los shards mejoran el rendimiento de las subconsultas generadas por Spider.
